

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 7
«Жадные алгоритмы»

Выполнил работу
Дышлевский Игорь
Академическая группа №J3111
Принято
Ментор, Владислав Вершинин

Санкт-Петербург
2024

1. Введение

Цель: решить задачу leetcode с помощью Жадных Алгоритмов.

Задачи:

- Найти задачу на leetcode
- Реализовать решение
- Провести тесты
- Построить графики скорости работы и сравнить с теоретической оценкой
- Написать отчет

2. Теоретическая подготовка

Типы данных:

- Long Long Int
- Vector

3. Реализация

Я выбрал задачу 2551 на Жадные алгоритмы.

2551. Put Marbles in Bags

Solved

Hard Topics Companies Hint

You have k bags. You are given a **0-indexed** integer array `weights` where `weights[i]` is the weight of the i^{th} marble. You are also given the integer k .

Divide the marbles into the k bags according to the following rules:

- No bag is empty.
- If the i^{th} marble and j^{th} marble are in a bag, then all marbles with an index between the i^{th} and j^{th} indices should also be in that same bag.
- If a bag consists of all the marbles with an index from i to j inclusively, then the cost of the bag is `weights[i] + weights[j]`.

The **score** after distributing the marbles is the sum of the costs of all the k bags.

Return the **difference** between the **maximum** and **minimum** scores among marble distributions.

Рисунок 3.1 Задача

Моё решение использует идею о том, что при разбиении массива весов на 2 части именно гранитные значения важны (потому что они определяют вес рюкзака), поэтому можно отсортировать массив сумм соседних элементов и для минимального разбиения взять $k - 1$ первых элементов, а для максимальной суммы $k - 1$ последних элементов. Таким образом получится две суммы: сумма минимальных по весу рюкзаков и максимальных по весу. Результатом будет разность этих сумм.

```

int put_marbles(std::vector<long long int>& weights, long long int k) {
    long long int n = weights.size();
    vector<long long int> sums(n - 1);
    for (long long int i = 0; i < n - 1; i++) {
        sums[i] = weights[i + 1] + weights[i];
    }
    quick_sort(sums, 0, sums.size() - 1);
    if (k == 1) {
        return 0;
    }
    else {
        long long int min_sum = 0, max_sum = 0;
        for (long long int i = 0; i < k - 1; i++) {
            min_sum += sums[i];
        }
        for (long long int i = n - 1 - (k - 1); i < sums.size(); i++) {
            max_sum += sums[i];
        }
        return max_sum - min_sum;
    }
}

```

Рисунок 3.2 Реализация алгоритма

Тесты были реализованы и успешно пройдены. Они вынесены в отдельную функцию с выводом результатов тестирования (OK/Error).

```

void test() {
    vector<long long int> arr = {1,3,5,1};
    long long int result = put_marbles(arr, 2);

    if (result == 4) {
        std::cout << "OK\n";
    } else {
        std::cout << "Test Failed\n";
    }

    arr = {1,3};
    result = put_marbles(arr, 2);

    if (result == 0) {
        std::cout << "OK\n";
    } else {
        std::cout << "Test Failed\n";
    }
}

```

Рисунок 3.3 Реализация функции теста

4. Экспериментальная часть

Подсчёт по памяти (только для циклов и сложных структур)

*Подсчеты приведены без учета веса самой структуры - только её содержимого

- $\text{vec}(\text{vector}<\text{long long int}> \text{arr}) = n * \text{size_of}(\text{long long int}) = n * 8 \text{ Байт}$
- $\text{vec}(\text{vector}<\text{long long int}> \text{sums}(n - 1)) = (n - 1) * \text{size_of}(\text{long long int}) = (n - 1) * 8 \text{ Байт}$

Подсчёт асимптотики (только для циклов и сложных структур).

- $O(N \cdot \log N)$ - асимптотика формируется сортировкой QuickSort, асимптотика остальной части меньше ($O(N)$)

График зависимости времени от числа элементов. Пример выполнения:

Теоретически заданная сложность задачи составляет $O(N \cdot \log N)$. Для тестирования алгоритма была собрана статистика, приведенная в таблице №1.

Таблица №1 - Подсчёт сложности реализованного алгоритма

Размер входного набора	10000	100000	1000000
Время выполнения программы, с	0.001712	0.0153	0.174594
$O(N \cdot \log N)$, с	0.00116	0.0145495	0.174594

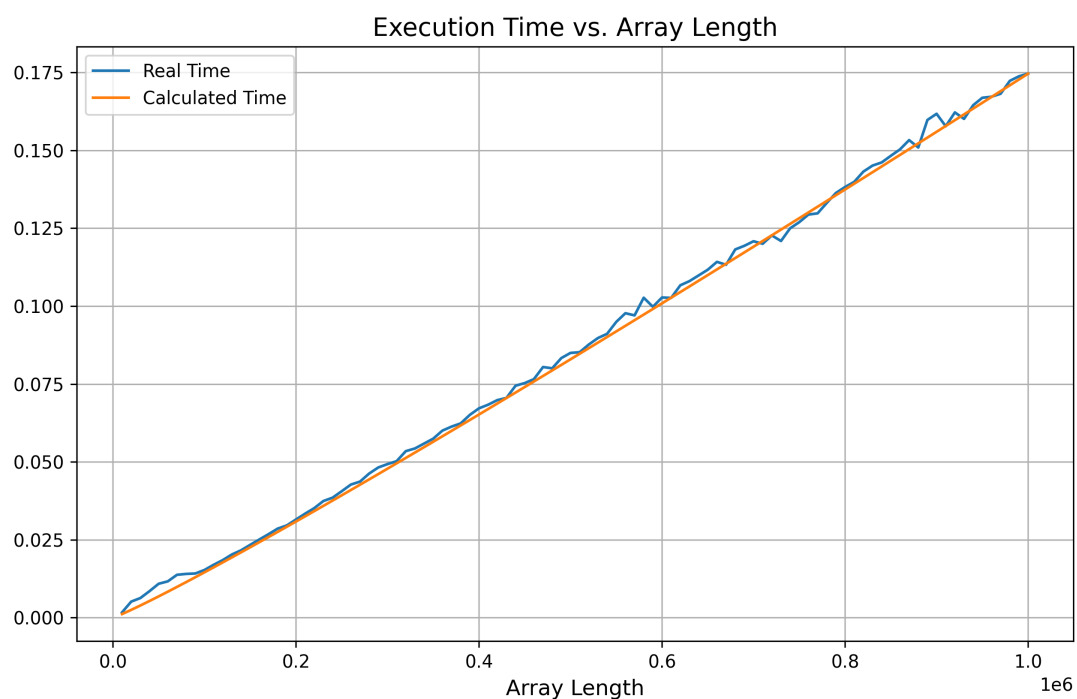


Рисунок 4.1 Теоретическое и реальное время

5. Заключение

В ходе работы были использованы Жадные алгоритмы для решения этой задачи (в данном случае жадный алгоритм заключается в локальном рассмотрении границ, работе с ними по отдельности и выборе локальных максимумов и минимумов).

6. Приложения

ПРИЛОЖЕНИЕ А

Листинг кода файла main7.cpp

```
#include <iostream>
#include <vector>
#include <random>
#include <ctime>

using namespace std;

void quick_sort(std::vector<long long int>& arr, long long int left, long long
int right) {
    if (left >= right) return;

    long long int pivot = arr[(left + right) / 2];
    long long int i = left, j = right;

    while (i <= j) {
        while (arr[i] < pivot) i++;
        while (arr[j] > pivot) j--;

        if (i <= j) {
```

```

        std::swap(arr[i], arr[j]);
        i++;
        j--;
    }
}
quick_sort(arr, left, j);
quick_sort(arr, i, right);
}

```

```

int put_marbles(std::vector<long long int>& weights, long long int k) {
    long long int n = weights.size();
    vector<long long int> sums(n - 1);
    for (long long int i = 0; i < n - 1; i++) {
        sums[i] = weights[i + 1] + weights[i];
    }
    quick_sort(sums, 0, sums.size() - 1);
    if (k == 1) {
        return 0;
    }
    else {
        long long int min_sum = 0, max_sum = 0;
        for (long long int i = 0; i < k - 1; i++) {
            min_sum += sums[i];
        }
        for (long long int i = n - 1 - (k - 1); i < sums.size(); i++) {
            max_sum += sums[i];
        }
        return max_sum - min_sum;
    }
}

```



```

}

void test_time(long long int len) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::vector<long long int> arr(len, 0);
    for (int i = 0; i < arr.size(); i++) {
        arr[i] = gen();
    }
    long long int k = gen() % len;
    clock_t start = clock();
    put_marbles(arr, k);
    clock_t end = clock();
    std::cout << "Time: " << (long double)(end - start) / (long double)
(CLOCKS_PER_SEC) << "\n";
}

void test() {
    vector<long long int> arr = {1,3,5,1};
    long long int result = put_marbles(arr, 2);

    if (result == 4) {
        std::cout << "OK\n";
    } else {
        std::cout << "Test Failed\n";
    }

    arr = {1,3};
    result = put_marbles(arr, 2);
}

```

```
    if (result == 0) {  
        std::cout << "OK\n";  
    } else {  
        std::cout << "Test Failed\n";  
    }  
}
```

```
int main() {  
    test();  
    return 0;  
}
```